

Databricks Fundamentals

Praktyczne wprowadzenie do platformy Databricks Lakehouse, od podstawowych pojęć, przez data engineering w PySpark i Delta Lake, po orkiestrację i governance. Dla osób, które chcą budować pipeline'y produkcyjne.

Agenda dnia

01

Wprowadzenie

Cele, uczestnicy, środowisko szkoleniowe

03

Architektura Apache Spark

Driver, executory, DAG, lazy evaluation, PySpark API

05

Delta Lake

ACID, MERGE, time travel, RESTORE, utrzymanie tabel

07

Unity Catalog i AI/BI

Governance, uprawnienia, dashboardy, Genie

02

Platforma Databricks

Workspace, compute, Unity Catalog, Volumes

04

Ingestia i architektura medalionowa

Volumes, Auto Loader, warstwy Bronze, Silver, Gold

06

Orkiestracja i Lakeflow

Lakeflow Jobs, harmonogram CRON, Repair Run, pipeline'y

08

Projekt końcowy: pipeline RetailHub

Kompletny pipeline zbudowany i uruchamiany jako job



Przerwy: ~10 min co ~90 min · Lunch ~13:00 · Pytania w każdej chwili · Q&A na końcu każdego bloku



Krzysztof Burejza

Resident Solution Architect @ Databricks

Resident Solution Architect w Databricks i data engineer z 10-letnim doświadczeniem w pracy z danymi, od SQL Server po platformy chmurowe takie jak Azure, Databricks i Microsoft Fabric. Zamienia surowe dane w wartościowe rozwiązania, buduje pipeline'y, hurtownie i lakehouse'y. Entuzjasta technologii na styku danych i sztucznej inteligencji, które czynią data engineering jeszcze ciekawszym.





Poznajmy się

Powiedzcie po kolei kilka słów o sobie:

- Imię
- Rola lub stanowisko
- Dotychczasowe doświadczenie z Databricks, jeśli jakieś jest

Dla kogo jest to szkolenie?

Grupa docelowa

- Analitycy danych
- Inżynierowie danych
- Specjaliści BI
- Osoby techniczne pracujące z pipeline'ami

Wymagania wstępne

- Podstawy SQL (SELECT, JOIN, GROUP BY)
- Znajomość baz relacyjnych
- Python nie jest wymagany, PySpark pokazujemy od zera

Czego nie zakładamy

Nie trzeba znać Databricks, Sparka ani storage w chmurze. Zaczynamy od podstaw i budujemy wiedzę krok po kroku.



Zasady szkolenia

Żeby szkolenie było skuteczne i komfortowe, umówmy się na kilka zasad:



Pytaj w każdej chwili

Śmiało przerywaj i pytaj. Twoje pytania pomagają całej grupie lepiej zrozumieć temat. Nie ma głupich pytań!



Laby we własnym tempie

Ćwiczenia robisz we własnym tempie. Jeśli skończysz wcześniej, zajrzyj do sekcji "Bonus Challenges" w notebookach. Pomóż też innym, jeśli potrzebują!



Błędy są mile widziane

Nie bój się pomyłek, to najlepsza droga do nauki. Od tego jest **time travel** w Delta Lake, który dziś poznamy!



Współpracuj

Omawiaj rozwiązania z sąsiadem, zanim uruchomisz kod. Wspólne rozwiązywanie problemów pogłębia zrozumienie tematu.



Korzystaj z asystenta AI

W Databricks masz wbudowanego asystenta AI. Skrót `Cmd+i` (lub `Ctrl+i` na Windows) pomoże w notebooku, zoptymalizuje kod i wyjaśni błędy.

Czego się dziś nauczysz

Platforma

Poruszanie się po środowisku Databricks, praca z compute, model Lakehouse

Spark

Model wykonania zapytań i transformacje w DataFrame API

Ingestia

Wczytywanie plików z Volumes, budowa warstw Bronze i Silver z jawnym schematem

Delta Lake

Transakcje ACID, MERGE, time travel, RESTORE, utrzymanie tabel

Orkiestracja

Tworzenie Lakeflow Jobs, harmonogram CRON, Repair Run dla tańszych powtórek

Projekt końcowy

Kompletny pipeline uruchamiany jako job produkcyjny, od Bronze do Gold

Jak działa środowisko szkoleniowe

→ Środowisko jest gotowe

Notebooki są skonfigurowane, compute już działa. Nic nie instalujesz, wystarczy otworzyć przeglądarkę.

→ Dwa języki, jeden notebook

Piszemy w **PySpark** i **SQL**, w tym samym notebooku. Przełączasz się magic commands.

→ Każdy blok: teoria → praktyka

Najpierw omawiamy temat na slajdach, potem od razu stosujesz go w notebooku. Uczymy się przez praktykę.

→ Natychmiastowy efekt

Wynik widzisz zaraz po uruchomieniu komórki. Błędy pokazują się od razu, debugowanie to część nauki.

SEKCJA 2

Platforma Databricks

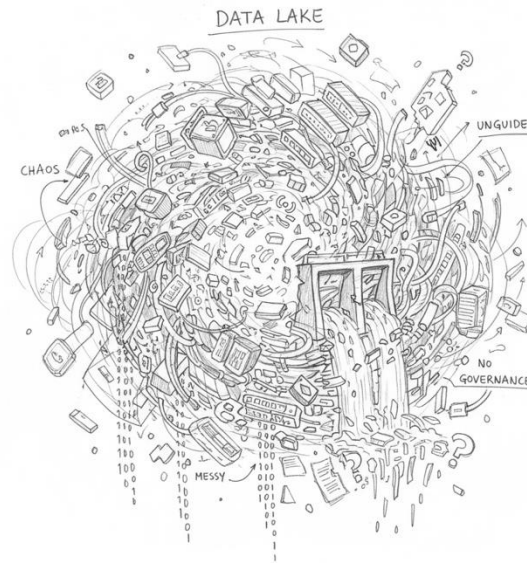
Zrozumienie platformy, zanim napiszemy pierwszą linię kodu.

Świat danych dziś, wyzwania



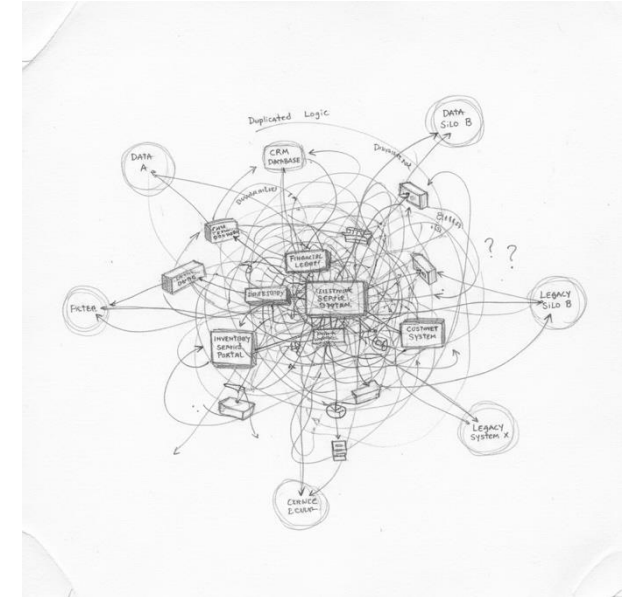
Klasyczne hurtownie danych

- Wysokie koszty licencji i infrastruktury
- Ograniczona skalowalność
- Sztywne schematy
- Głównie dane ustrukturyzowane



Data Lake

- Tani storage, ale grozi "data swamp"
- Brak transakcji ACID (atomowość, spójność, izolacja, trwałość)
- Problemy z zarządzaniem (data governance)
- Trudne wyszukiwanie danych i lineage



Rozproszone środowiska

- Osobne systemy dla analityki i machine learningu
- Duplikacja danych i logiki
- Silosy organizacyjne
- Problemy z wydajnością przy skali

Wpływ sztucznej inteligencji i dużych modeli językowych na wymagania wobec platform danych



→ Dane nieustrukturyzowane

Modele AI potrzebują tekstu, obrazów, audio i wideo. To znacznie więcej niż klasyczne tabele.

→ Jedna platforma

Jedno miejsce na dane surowe, cechy, modele i metadane. Koniec z kopiowaniem między systemami.

→ Governance i jakość

AI i LLM-y mnożą problemy z jakością danych. Rośnie rola lineage, audytu i kontroli dostępu.

→ Zbieżność hurtowni, data lake i machine learningu

Potrzebna jest platforma łącząca możliwości hurtowni, elastyczność data lake i wsparcie dla AI w jednym.

Era generatywnej AI zmienia oczekiwania wobec platform danych i wymusza konsolidację oraz modernizację architektury.

Databricks: Platforma Data + AI

Czym jest Databricks?

Databricks to platforma analityczna zbudowana na Apache Spark przez twórców tego projektu z UC Berkeley. Daje kompletne rozwiązanie klasy lakehouse, łączące:

- Data engineering, czyli przetwarzanie i przygotowanie danych
- Data science, czyli zaawansowaną analitykę i machine learning
- Analitykę biznesową, czyli wizualizacje i raporty

Platforma działa we wszystkich głównych chmurach (AWS, Azure, GCP), więc wdrożenie pasuje do posiadanej infrastruktury.



databricks



Początki Databricks

2013: powstanie Databricks

Założyciele, twórcy Apache Spark z UC Berkeley, w tym Matei Zaharia, Ali Ghodsi i Ion Stoica, postanawiają skomercjalizować technologię.

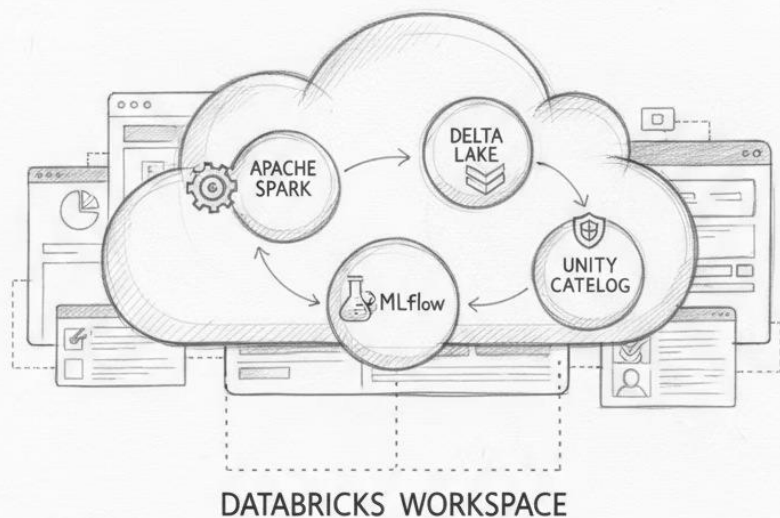
Misja firmy

Uprościć korzystanie z Apache Spark w produkcji i w chmurze. Udostępnić zaawansowaną analitykę szerszemu gronu organizacji.

Databricks założył zespół, który nie tylko stworzył Sparka, ale nadal rozwija zarówno silnik, jak i cały ekosystem wokół niego.

Elementy platformy

Databricks to chmurowa platforma Data + AI zbudowana na technologiach open source:



Apache Spark

Rozproszone przetwarzanie w pamięci, standardowy silnik analityczny dla dużych zbiorów danych.

Delta Lake

Otwarty format tabel z transakcjami ACID i time travel, fundament Lakehouse.

MLflow

Otwarta platforma do zarządzania cyklem życia modeli ML, od eksperymentu po wdrożenie.

Unity Catalog

Centralny katalog metadanych z governance, śledzeniem lineage i kontrolą dostępu.

Dostępny w głównych chmurach, w tym jako Azure Databricks, natywnie zintegrowany z ekosystemem Microsoft Azure.

Architektura Databricks: control plane i data plane

Control plane

Zarządzane przez Databricks:

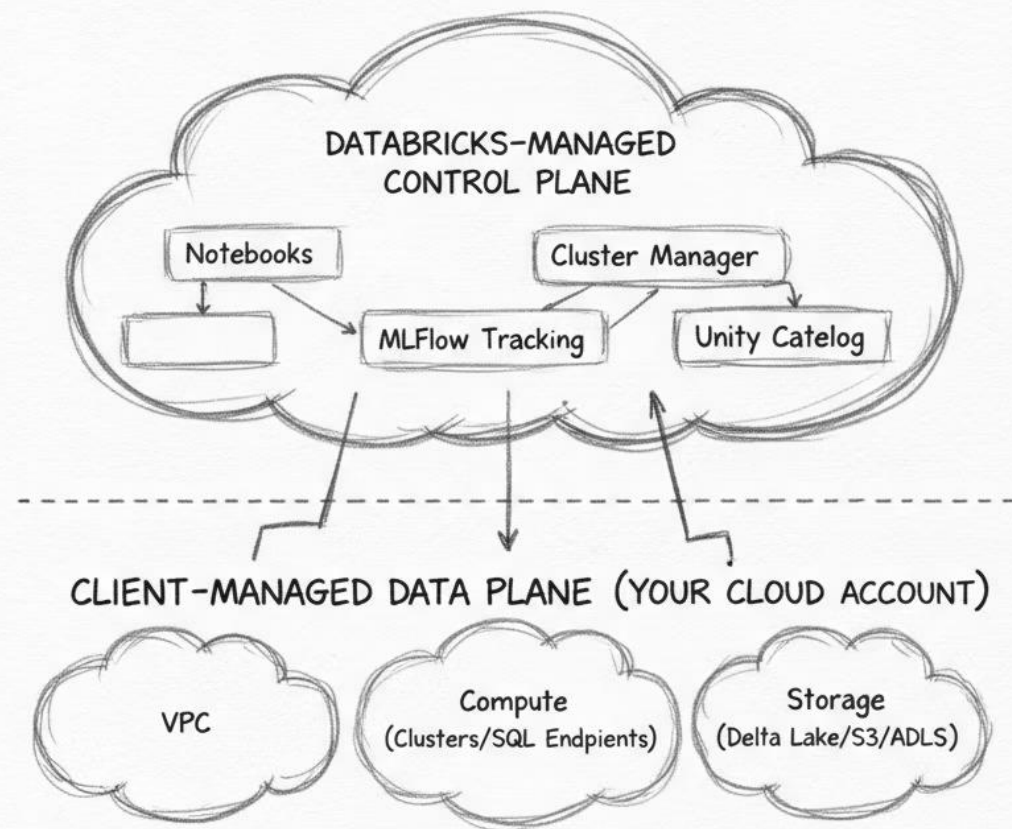
- Aplikacja webowa, interfejs
- Cluster manager, zarządzanie compute
- Job scheduler, harmonogram zadań
- Notebook server, środowisko pracy
- Security i governance
- Metastore, metadane

Data plane

Środowisko klienta:

- Klastry compute (Apache Spark)
- Storage klienta (S3/ADLS/GCS)
- Sieć (VPC/VNet)
- Grupy i polityki bezpieczeństwa

Przy compute serverless sam klaster działa po stronie Databricks, a dane zostają w Twoim storage.



Architektura Databricks

Lakehouse w skrócie

Warstwa storage

Object storage (ADLS/S3/GCS), tani, skalowalny, trwały. Wszystkie dane w jednym miejscu, bez duplikacji.

Warstwa tabel

Tabele Delta, Unity Catalog, metadane. Warstwa abstrakcji dająca strukturę i transakcyjność.

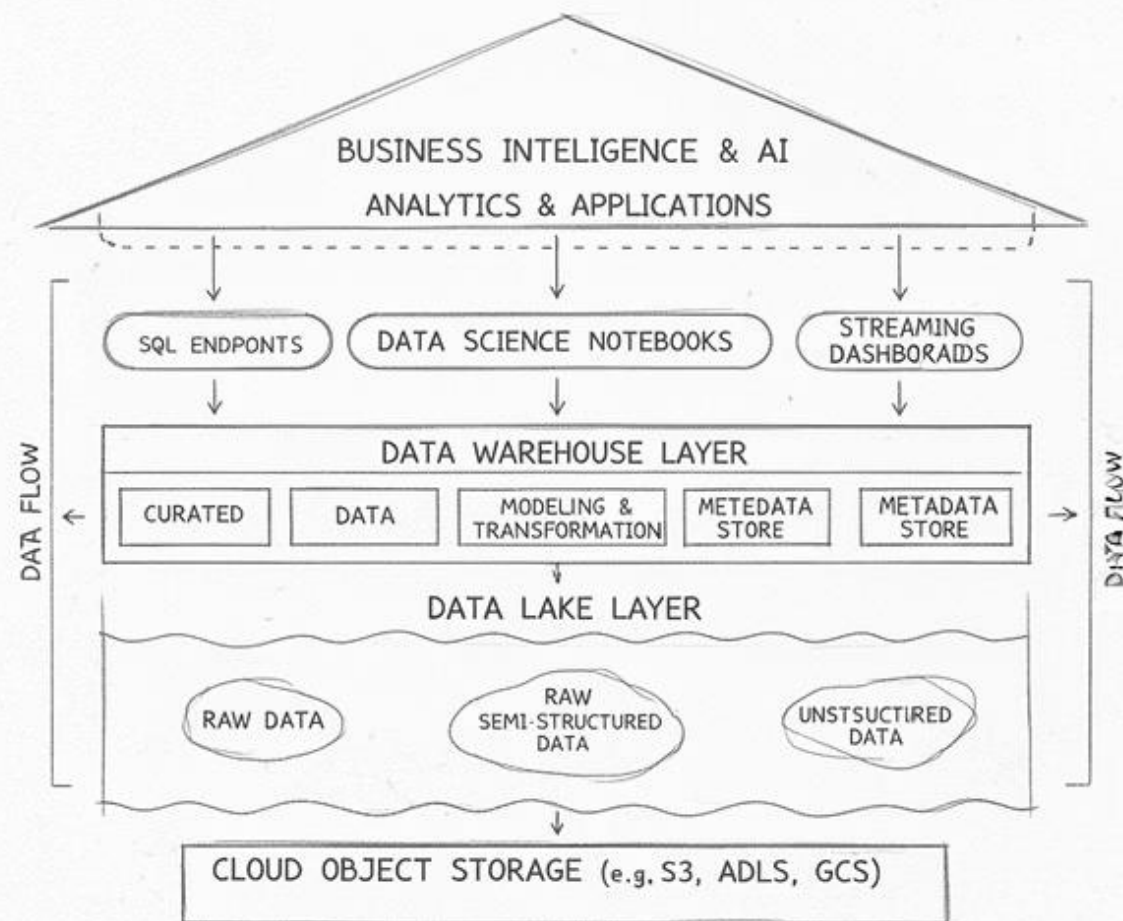
Warstwa compute

Klastry Apache Spark, SQL warehouses z Photonem. Rozproszony silnik, który skaluje się elastycznie.

Warstwa governance

Unity Catalog, polityki bezpieczeństwa, lineage. Jednolite zarządzanie dostępem i audyt całej platformy.

LAKEHOUSE ARCHITECTURE



Integracja Databricks z chmurą

Storage danych

Natywna integracja z Azure Data Lake Storage, AWS S3 i Google Cloud Storage. Databricks czyta i zapisuje dane wprost w chmurze, bez pośredników.

Bezpieczeństwo i tożsamość

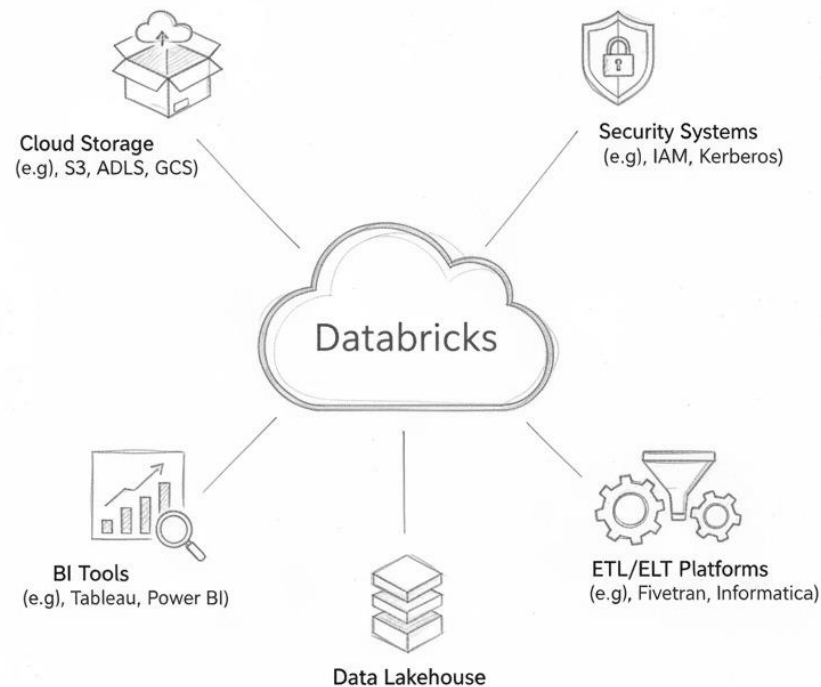
Microsoft Entra ID, AWS IAM, Google Cloud IAM do uwierzytelniania. SCIM i SSO dla firm. Szyfrowanie w spoczynku i w transporcie.

Integracje BI

Konektory do Power BI, Tableau, Looker i Qlik. Sterowniki JDBC/ODBC dla dowolnych narzędzi SQL. Partner Connect na szybki start.

Integracje ETL/ELT

Współpraca z Fivetran, dbt, Azure Data Factory, Informatica i Talend. Auto Loader do przyrostowego ładowania plików ze storage.



Główne elementy platformy



Workspace

Twoje środowisko pracy: notebooki, repozytoria Git, pipeline'y i joby, wszystko w jednym miejscu.



Compute

Compute serverless albo klasyczne klastry Spark. Płacisz za DBU × czas, więc bezczynny klaster to strata pieniędzy.



Storage

Dane leżą w storage w chmurze (ADLS, S3, GCS), a sięgasz po nie przez tabele i Volumes w Unity Catalog.



Unity Catalog

Centralny katalog metadanych i warstwa governance dla całej organizacji: tabele, uprawnienia, lineage, audyt.



Delta Lake

Warstwa transakcyjna nad plikami Parquet. Domyślny format tabel w Databricks, wnosi ACID do data lake.

Jak odnaleźć się w workspace

Notebooki

Główne miejsce pracy. Komórki w Pythonie, SQL, Scali lub R, wyniki i wykresy od razu, komentarze i wspólna edycja na żywo.

SQL editor i dashboardy

Piszesz zapytania na SQL warehouse, zapisujesz je i robisz z wyniku dashboard AI/BI bez opuszczania platformy.

Catalog Explorer

Przeglądasz katalogi, schematy, tabele i volumes. Podgląd danych, schemat, uprawnienia, lineage i historia w jednym miejscu.

Git folders

Podłączasz workspace do repozytorium Git, pracujesz na branchu, commitujesz i otwierasz pull requesty z poziomu Databricks.

Joby i pipeline'y

Lakeflow Jobs do orkiestracji i Lakeflow pipelines do deklaratywnego ETL. Oba znajdziesz w menu po lewej.

Databricks Assistant

Wbudowany asystent AI. Wyjaśnia błędy, pisze i poprawia kod, odpowiada na pytania o Twoje tabele. Skrót Cmd+i lub Ctrl+i.

Unity Catalog, czyli gdzie mieszkają dane

-- Każdy obiekt ma nazwę trzypoziomową

```
SELECT * FROM retail_prod.bronze.orders; -- katalog . schemat . tabela
```

```
SELECT * FROM read_files('/Volumes/retail_prod/bronze/landing/orders.json');
```

Katalog

Najwyższy poziom, zwykle jeden na środowisko lub domenę: dev, test, prod. Tu zaczyna się izolacja i uprawnienia.

Schemat

Grupa powiązanych obiektów w katalogu, u nas bronze, silver i gold.

Tabela, widok, volume

Tabele i widoki trzymają dane ustrukturyzowane. Volumes trzymają pliki: JSON, CSV, PDF, obrazy, artefakty modeli.

📄 Volumes zastępują mounty DBFS. Ścieżka zaczyna się zawsze od `/Volumes/<katalog>/<schemat>/<volume>/` i podlega uprawnieniom Unity Catalog.

Compute, rodzaje i dobre praktyki

Serverless

Startuje w sekundy, zawsze najnowszy runtime, nic nie konfigurujesz. Domyślnie dostępny w workspace z UC.

All-purpose cluster

Praca interaktywna w notebookach, współdzielony. Elastyczny, ale droższy, ustaw auto-terminate na 10 min.

Job cluster

Tworzony automatycznie na czas joba i kasowany po nim. Tani i izolowany, klasyk dla produkcji.

SQL warehouse

Dedykowany compute dla SQL i dashboardów. Zoptymalizowany pod zapytania BI i wielu użytkowników.

Autoskalowanie

Klasyczny klaster dobiera liczbę workerów do obciążenia. Serverless skaluje się sam i liczy koszt tylko w czasie pracy.

Wersje runtime

Klasyczny klaster ma przypisany Databricks Runtime. Dla stabilności wybierz LTS, na przykład DBR 18 LTS ze Sparkiem 4.1, dla nowości DBR 19.

Jak trzymać koszty compute w ryzach

Compute rozliczasz w DBU razy czas, plus maszyny w chmurze pod spodem. Cztery nawyki trzymają rachunek w ryzach:

Dobierz compute do zadania

Nie uruchamiaj pipeline'u z harmonogramu na all-purpose cluster. Serverless lub job cluster startuje na żądanie i gaśnie po zadaniu.

Auto-terminate i auto-stop

Każdy klaster interaktywny potrzebuje timeoutu, zwykle 10 do 30 minut. SQL warehouse zatrzyma się sam po kilku minutach bezczynności.

Cluster policies

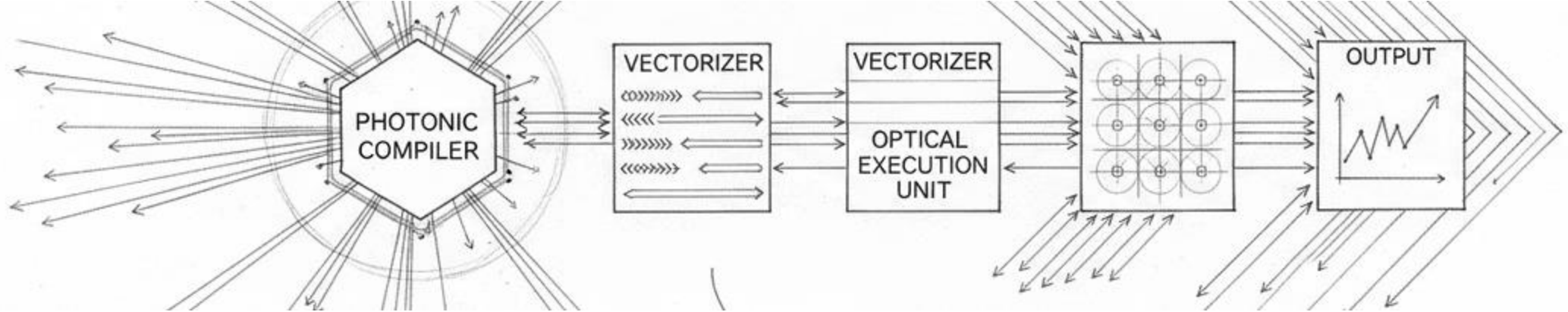
Administrator ustala polityki ograniczające typy maszyn, liczbę workerów i runtime, żeby nikt nie wstał z 40 węzłami do pliku 10 MB.

Taguj i monitoruj

Oznaczaj joby i klastry nazwą zespołu lub projektu, a potem śledź wydatki w dashboardach i tabelach systemowych.

Rozmiar pod dane, nie pod wygodę

Zacznij mało i zwiększaj dopiero, gdy etap faktycznie zwalnia. Photon i Spark UI pokażą, gdzie ucieka czas.



Czym jest Photon

Photon to natywny, wysokowydajny silnik zapytań stworzony przez Databricks:

Wykonanie wektorowe

Operacje na kolumnach zamiast wiersz po wierszu. Instrukcje SIMD wyciskają maksimum z CPU.

Zgodność

Pełna zgodność ze Spark SQL i DataFrame API. Nie zmieniasz kodu, Photon włączasz jednym przełącznikiem.

Cel biznesowy

Szybsze zapytania analityczne przy niższym koszcie compute. Lepszy TCO dla obciążeń SQL i BI.

SEKCJA 3

Architektura Apache Spark

Zrozumienie, jak Spark wykonuje kod, to klucz do wydajnych i skalowalnych pipeline'ów.

Model wykonania w Spark: driver i executory

Driver

Jeden węzeł nadrzędny, który kompiluje kod, buduje DAG operacji i rozdziela zadania na workery. Tu działa cała logika sterująca.

Uwaga

Nie wołaj `.collect()` na dużym DataFrame, wciąga wszystko do pamięci drivera i może go zabić.

Executory

Od jednego do N węzłów, które wykonują zadania i trzymają dane pośrednie w pamięci. Jedno zadanie to dokładnie jedna partycja.

Partycje

Podstawowa jednostka równoległości w Spark. Więcej partycji = więcej równoległych zadań. Dane nie opuszczają klastra do czasu akcji.

Shuffle

Najdroższa operacja w pipeline, przenosi dane między partycjami przez sieć. Im mniej shuffle, tym lepsza wydajność.

Lazy evaluation: transformacje i akcje

Transformacje (lazy)

Budują plan wykonania, **nic się jeszcze nie liczy**. Na przykład `select`, `filter`, `withColumn`, `join`, `groupBy`. Spark zapisuje co zrobić, nie kiedy.



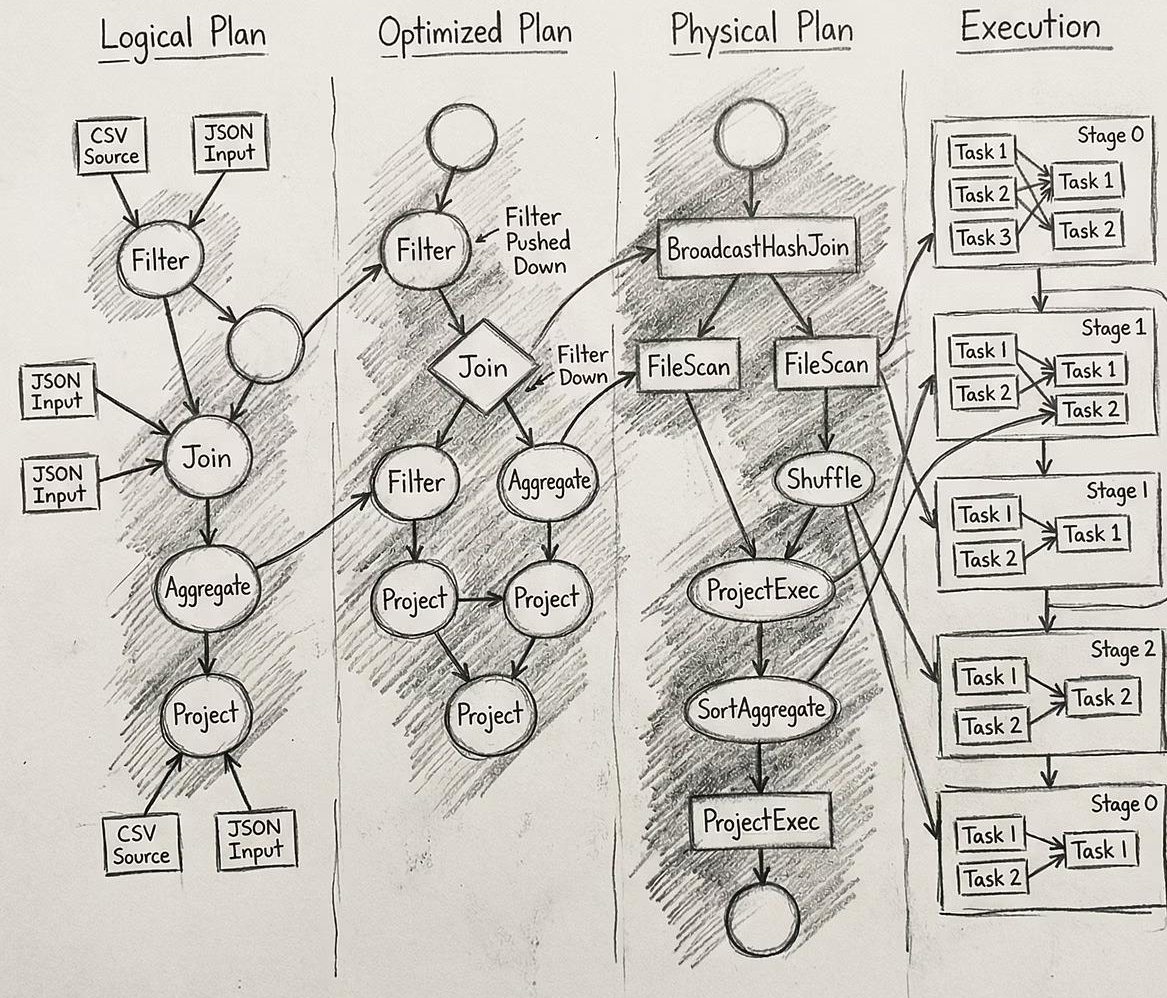
Akcje (eager)

Uruchamiają cały plan. Na przykład `count`, `show`, `display`, `write`, `collect`. Dopiero teraz Spark sięga po dane.

- 📌 **Główna korzyść:** przed uruchomieniem **Catalyst Optimizer** analizuje cały łańcuch transformacji i przepisuje go na najefektywniejszy plan fizyczny, zmieniając kolejność filtrów, usuwając zbędne kolumny i przesuwając predykaty jak najbliżej źródła.

DAG i plan wykonania

Każdy job w Spark to graf skierowany acykliczny (DAG), czyli ciąg kroków od źródła do wyniku. Catalyst Optimizer stosuje dwie kluczowe optymalizacje: **predicate pushdown** (filtry blisko źródła) i **column pruning** (czytanie tylko potrzebnych kolumn). Potem **Adaptive Query Execution** przeplanowuje job w trakcie działania na podstawie realnych statystyk, naprawiając skośne joiny i za duże shuffle. Wąskie gardła znajdziesz w **Spark UI**, zakładki Jobs, Stages i SQL.



PySpark DataFrame API, jak czytać kod

Model DataFrame

Rozproszona tabela o niezmiennym schemacie. Operacje łączymy w łańcuch:

```
df.filter(...).withColumn(...).groupBy(...).agg(...)
```

Odwołania do kolumn

Używaj `col("name")` w samodzielnych wyrażeniach, a `df["name"]` gdy w joinie dwa DataFrame mają kolumnę o tej samej nazwie.

display() a show()

`display(df)` to funkcja Databricks, rysuje tabelę ze stronicowaniem. `df.show(n, truncate=False)` to standard Sparka, działa też poza Databricks.

SQL w kodzie

`spark.sql("SELECT ...")` wykonuje SQL i zwraca DataFrame, więc możesz dalej łączyć transformacje.

SEKCJA 4

Ingestia i architektura medalionowa

Podział data lake na warstwy jakości to fundament wiarygodnych pipeline'ów, które da się audytować.

Architektura medalionowa, po co trzy warstwy?

🥇 Gold

Agregaty gotowe dla biznesu. Jeden wiersz = jeden wymiar biznesowy. Zasilają raporty, dashboardy i analityków.

🥈 Silver

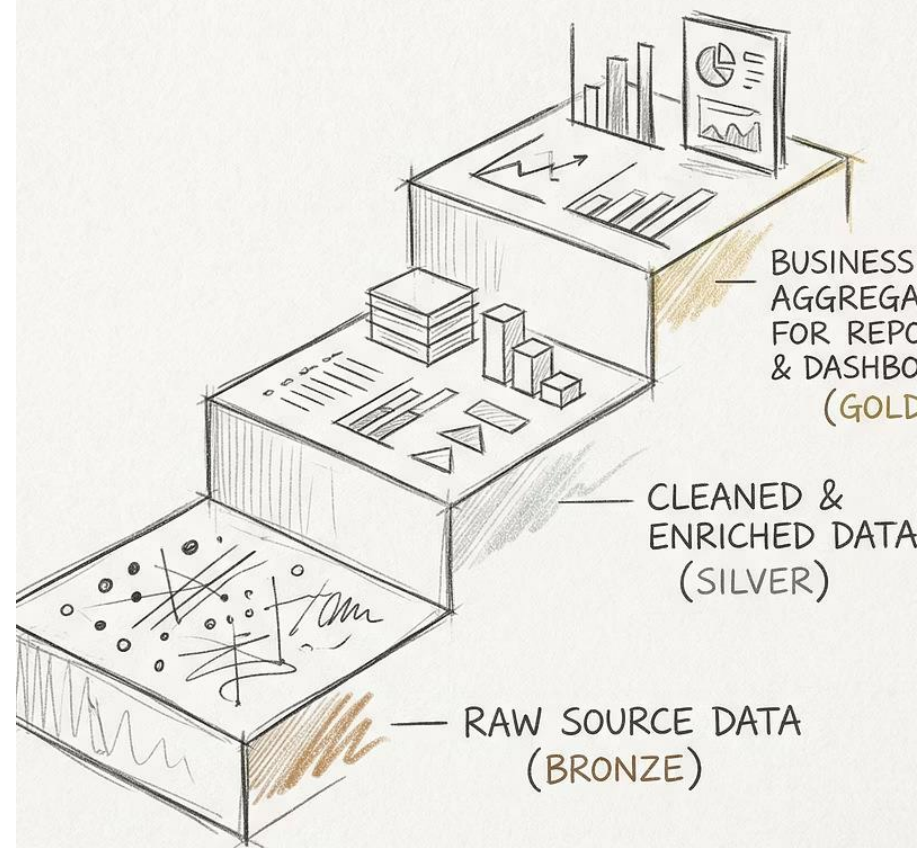
Dane oczyszczone, otypowane i wzbogacone. Jakość zagwarantowana, gotowe do analiz.

🥉 Bronze

Dane surowe, dokładnie takie jak ze źródła. Bez filtrów i logiki biznesowej. Niezmienny zapis prawdy.

Każda warstwa to osobna tabela Delta, którą można odpytać, zaudytować i przywrócić. Błąd w Silver nie psuje Bronze. Kolejne warstwy zawsze przeliczysz od nowa ze źródła.

DATA REFINEMENT STAGES



Jak dane trafiają do platformy

Przed warstwami medalionu zawsze jest ingestia. Databricks daje kilka poziomów automatyzacji:

Pliki w volume

Punkt startowy na tym szkoleniu. Pliki lądują w `/Volumes/<katalog>/<schemat>/<volume>/`, czytasz je przez `spark.read` lub `read_files()` i zapisujesz tabelę Bronze. Prosto i w pełni ręcznie.

Auto Loader

Przyrostowe ładowanie plików. `cloudFiles` pamięta, co już przetworzył, więc ponowny bieg bierze tylko nowe pliki. Obsługuje wykrywanie i ewolucję schematu oraz rescued data.

Lakeflow Connect

Zarządzane konektory do baz i aplikacji SaaS, na przykład Salesforce, Workday czy SQL Server. Konfiguracja zamiast kodu, z obsługą CDC.

Streaming tables

Tabela zdefiniowana zapytaniem strumieniowym. Dociąga nowe dane na bieżąco lub z harmonogramu i jest podstawą pipeline'ów deklaracyjnych.

Warstwa Bronze, zasady

Złota zasada

Surowe jak jest: ładuj dokładnie to, co przyszło ze źródła. Bez filtrów, bez logiki biznesowej, bez interpretacji typów.

Wszystkie kolumny jako StringType

JSON i CSV nie mają typów. Bronze zachowuje surowe stringi, a typy nadajesz w Silver, gdzie panujesz nad logiką.

Obowiązkowe kolumny metadanych

- `_load_ts`, czyli `current_timestamp()`, kiedy plik został wczytany
- `_source_file`, czyli `input_file_name()`, z jakiego pliku pochodzi wiersz

Zawsze jawny schemat

Nie używaj `inferSchema` na produkcji. Powoduje podwójny odczyt danych i niczego nie gwarantuje. Zdefiniuj `StructType` jawnie.

Format docelowy

Delta, bo od pierwszego dnia daje ACID, kontrolę schematu i time travel.

Jawny schemat: StructType i StructField

```
from pyspark.sql.types import StructType, StructField, StringType
```

```
schema = StructType([  
    StructField("order_id", StringType(), nullable=False),  
    StructField("customer_id", StringType(), nullable=True),  
    StructField("total_amount", StringType(), nullable=True),  
    StructField("order_datetime", StringType(), nullable=True),  
    # ... pozostałe pola  
])
```

nullable=False na kluczach

Brak klucza wyłapiesz przy odczycie.
Lepiej, żeby zadanie padło od razu, niż
żeby nulle szły dalej przez pipeline.

Dlaczego StringType w Bronze?

JSON i CSV nie niosą informacji o
typach. String jest bezpiecznym
domyślnym wyborem, a rzutowanie
robisz w Silver, z pełną kontrolą.

Unikaj inferSchema

Spark czyta dane dwa razy i może źle
zgadnąć. Zgadywane typy to dług
techniczny, który po cichu psuje pipeline.

Warstwa Silver, czyszczenie i typy

01

Nadaj właściwe typy

Zamień StringType na typy docelowe: `TimestampType`, `DoubleType`, `IntegerType`. Zawsze podawaj format:
`to_timestamp(col("order_datetime"), "yyyy-MM-dd'T'HH:mm:ss")`

03

Odfiltruj błędne wiersze

Usuń wiersze z nullami w kluczach. Silver to obietnica jakości, dalszy kod nie powinien bronić się przed nullami.

02

Dodaj kolumny wyliczane

Dołącz kolumny istotne biznesowo: `net_amount` (kwota po rabacie), `order_tier` (segment wartości). To miejsce dla nich, nie Gold.

04

Usuń metadane techniczne

Wyrzuć kolumny w rodzaju `_source_file`, potrzebne w Bronze, ale zbędne dalej. Silver ma być czysty i biznesowy.

📌 Tryb zapisu: `mode("overwrite")` plus `option("overwriteSchema", "true")`, gdy zmiana schematu jest zamierzona.

Jakość danych: constraints i expectations

Silver obiecuje jakość, więc obietnicę trzeba egzekwować w kodzie, a nie na wiki:

```
-- Constraint na tabeli: zły zapis się nie uda
ALTER TABLE silver.orders
ADD CONSTRAINT positive_amount
CHECK (net_amount >= 0);
```

```
# Expectation w pipeline: warn, drop albo fail
@dp.expect_or_drop(
    "valid_order_id", "order_id IS NOT NULL")
def silver_orders(): ...
```

Warn

Wiersz trafia do tabeli, a naruszenie ląduje w metrykach pipeline. Dobre przy obserwacji nowego źródła.

Drop

Wiersz jest odrzucany i zliczany. Tabela zostaje czysta, a bieg trwa dalej.

Fail

Bieg zatrzymuje się od razu. Zostaw to dla reguł, których złamać nie wolno, jak brak klucza głównego.

Constraints chronią tabelę, expectations chronią pipeline. Jedno i drugie daje liczby, które wrzucisz na dashboard.

Najczęstsze transformacje PySpark, ściągą

Operacja	Kod
Rzutowanie typu	<code>col("x").cast(DoubleType())</code>
Parsowanie daty	<code>to_timestamp(col("x"), "yyyy-MM-dd'T'HH:mm:ss")</code>
Wartość warunkowa	<code>when(condition, value).otherwise(other)</code>
Zaokrąglenie	<code>round(col("x"), 2)</code>
Stała	<code>lit(0)</code>
Wyciągnięcie daty	<code>to_date(col("ts"))</code>
Filtr nulli	<code>col("x").isNotNull()</code>
Usunięcie kolumny	<code>.drop("col_name")</code>

Warstwa Gold, agregaty biznesowe

Jeden wiersz = jeden wymiar biznesowy

Tabele Gold są do bezpośredniego użycia. Jeden wiersz to komplet danych dla jednego dnia, kategorii albo regionu, a nie pojedyncza transakcja.

Zawsze przeliczane z Silver

W Gold używaj `mode("overwrite")` bez `overwriteSchema`. Schemat musi być stabilny między biegami, bo zależą od niego raporty.

Typowy wzorzec agregacji

```
df_gold = df_silver.groupBy("order_date").agg(
    count("*").alias("order_count"),
    sum("net_amount").alias("total_revenue"),
    round(avg("net_amount"), 2)
    .alias("avg_order_value")
)
```

Licz na `net_amount`, nie `total_amount`

Agreguj kwotę po rabacie. `total_amount` to wartość sprzed rabatu z Bronze, użyta w Gold zawyżyłaby przychód.

SEKCJA 5

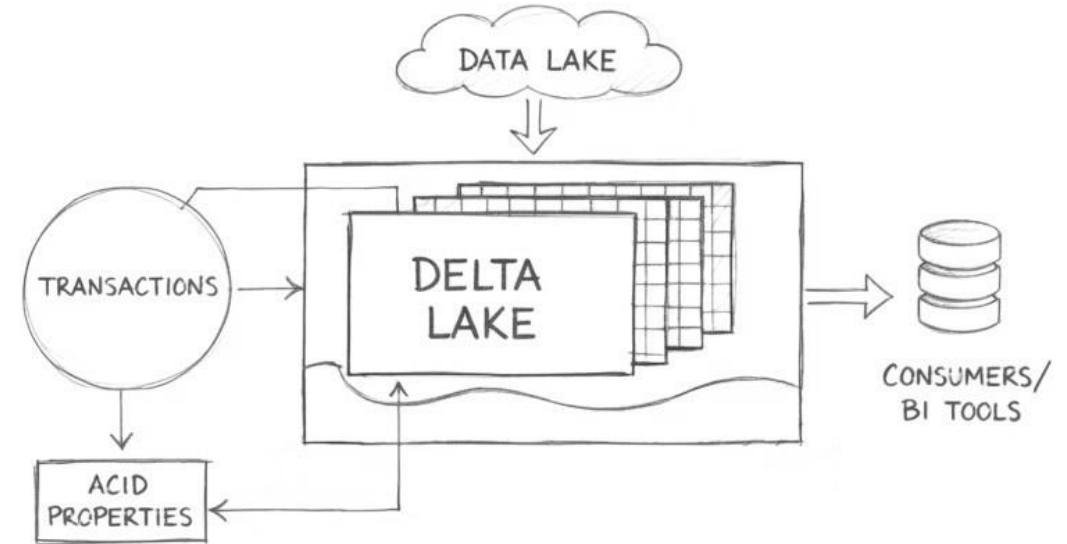
Delta Lake

Delta Lake zamienia zwykły storage plików w wiarygodną, transakcyjną platformę danych.

Czym jest Delta Lake?

Delta Lake to otwarta warstwa storage, która wnosi niezawodność do data lake. Łączy najlepsze cechy baz danych i data lake, dając:

- **Transakcje ACID:** atomowość, spójność, izolacja i trwałość operacji, czyli gwarancja integralności danych.
- **Kontrola schematu:** blokuje zapisy niezgodne ze schematem tabeli i tym samym pilnuje jakości danych.
- **Wersjonowanie danych:** daje dostęp do historycznych wersji tabel, ułatwia audyt, wycofanie zmian i odzyskiwanie danych.
- **Skalowalne metadane:** obsługuje tabele z miliardami plików i partycji.
- **Streaming i batch razem:** ten sam data lake obsługuje przetwarzanie wsadowe i czasu rzeczywistego.



Tabele managed i external

Każda tabela w Unity Catalog jest jednego z dwóch rodzajów, a wybór decyduje, kto dba o pliki:

Tabela managed

Unity Catalog jest właścicielem danych i metadanych oraz sam wybiera lokalizację. To typ domyślny i rekomendowany.

Dostajesz automatyczne utrzymanie przez predictive optimization, tańszy storage i sprzątanie plików po usunięciu tabeli.

```
CREATE TABLE silver.orders (...)
```

Tabela external

Wskazujesz Databricks lokalizację w swoim storage przez external location. Databricks zarządza wtedy tylko metadanymi.

Używaj, gdy te same pliki zapisuje inny silnik, przy migracji albo gdy polityka retencji wymusza konkretną ścieżkę. Bez predictive optimization i bez sprzątania plików.

```
CREATE TABLE ... LOCATION 'abfss://...'
```

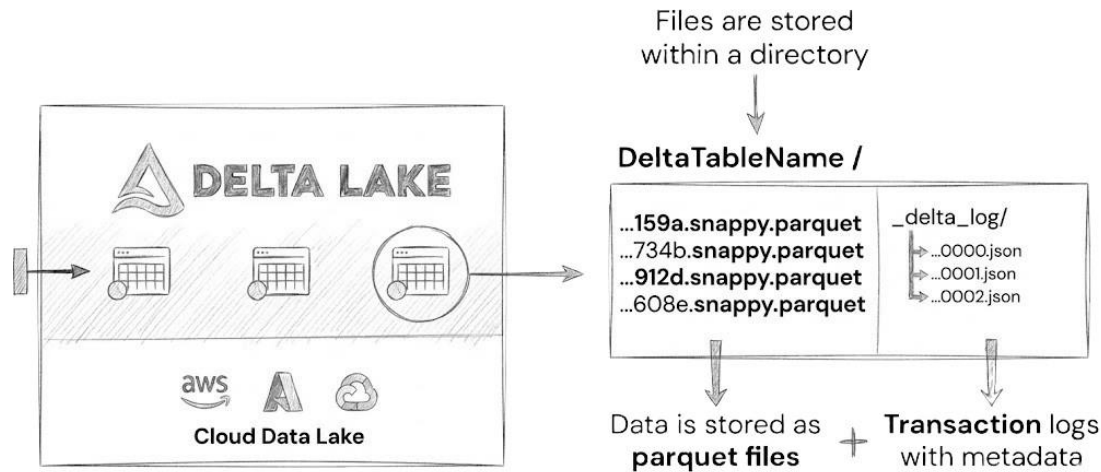
Delta log, serce transakcyjności

Delta log to uporządkowany ciąg plików JSON w katalogu `_delta_log`. Każdy plik to jedna transakcja z kompletem informacji o zmianach w tabeli.

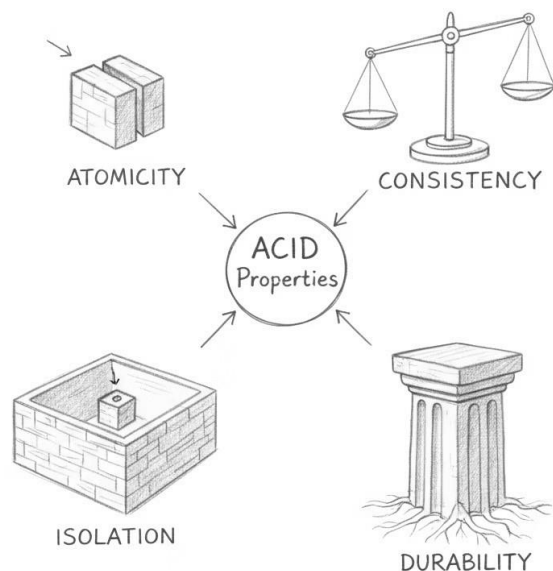
Co zawiera wpis w delta logu:

- Listę dodanych plików Parquet wraz z metadanymi
- Listę plików usuniętych
- Operacje na schemacie (dodanie lub usunięcie kolumn)
- Metadane operacji (czas, użytkownik, rodzaj)
- Statystyki danych (min i max dla data skipping)

Każdy odczyt zaczyna się od przeczytania delta logu, żeby ustalić aktualny stan tabeli i listę aktywnych plików.



Własności ACID w data lake



Atomowość

Operacje są atomowe, albo kończą się w całości, albo są wycofywane. Żadnych częściowych zapisów i niespójnych stanów.

Spójność

Kontrola schematu pilnuje, żeby po każdej transakcji tabela była zgodna ze zdefiniowanym schematem.

Izolacja

Izolacja serializowalna, równoległe operacje sobie nie przeszkadzają. Czytający zawsze widzi spójny snapshot.

Trwałość

Zatwierdzone transakcje są trwałe i przetrwają awarię dzięki delta logowi zapisanemu w object storage.

MERGE, upsert w jednym poleceniu

```
MERGE INTO silver.orders AS target
USING updates AS source
  ON target.order_id = source.order_id
WHEN MATCHED AND source.status = 'cancelled' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Idempotentny z natury

Ponowny bieg tej samej paczki aktualizuje pasujące wiersze zamiast je duplikować. Dzięki temu pipeline można bezpiecznie powtórzyć.

UPDATE, DELETE, INSERT

Delta obsługuje pełny zestaw CRUD. `UPDATE` i `DELETE` przepisują tylko potrzebne pliki, wspierane przez deletion vectors.

Pilnuj warunku złączenia

Mało selektywny warunek przepisze całą tabelę. Ogranicz źródło i sklastruj tabelę docelową po kluczu merge.

DESCRIBE HISTORY, audyt transakcji

```
DESCRIBE HISTORY bronze_transactions;
```

-- Wynik pokazuje pełną historię operacji:

version	timestamp	operation	operationParameters
0	2024-01-15 10:00:00	CREATE	{mode: ErrorIfExists}
1	2024-01-15 11:30:00	WRITE	{mode: Append}
2	2024-01-15 14:00:00	MERGE	{predicate: ["id"]}
3	2024-01-15 16:45:00	DELETE	{predicate: ["status='invalid'"]}
4	2024-01-15 18:00:00	OPTIMIZE	{predicate: []}

Polecenie `DESCRIBE HISTORY` pokazuje pełny ślad audytowy operacji na tabeli Delta. Każda wersja to niezmienny snapshot, przydatny przy time travel, analizie powłamaniowej i odtwarzaniu po awarii.

Historię transakcji zarządza Delta Lake, a czas jej przechowywania ustala konfiguracja retencji, domyślnie 30 dni.

Kontrola schematu a ewolucja schematu

Kontrola schematu (domyślnie włączona)

Zapis danych o schemacie niezgodnym z tabelą kończy się `AnalysisException`. To chroni produkcję, bo złe dane głośno padają, zamiast po cichu psuć tabelę.

Stosuj w: każdym pipeline produkcyjnym.

Ewolucja schematu

Dodaj `option("mergeSchema", "true")`, żeby nowe kolumny same dopisały się do schematu tabeli. Istniejące kolumny zostają bez zmian.

Stosuj w: migracjach, stopniowym dodawaniu pól, na dev.

`overwriteSchema`

Używany z `mode("overwrite")`, całkowicie podmienia schemat. Stosuj świadomie, dla odbiorców danych to zmiana łamiąca zgodność.

Time travel

Podróż w czasie po historii danych

Time travel, odczyt historyczny

Składnia zapytań historycznych

```
-- Odczyt po numerze wersji
SELECT * FROM table_name VERSION AS OF 0;

-- Odczyt po znaczniku czasu
SELECT * FROM table_name
TIMESTAMP AS OF '2024-01-15 10:00:00';

-- W API DataFrame
df = spark.read \
    .format("delta") \
    .option("versionAsOf", 0) \
    .load(path)
```

Time travel to wyróżnik Delta Lake, pozwala czytać dane z dowolnego momentu historii tabeli. Każda wersja to niezmienny snapshot dostępny zgodnie z polityką retencji.

Zastosowania biznesowe

- Audyt i zgodność regulacyjna
- Analiza zmian danych w czasie
- Odtwarzanie dawnych raportów
- Porównanie wersji przed i po
- Testowanie logiki transformacji
- Odzyskanie danych po błędnej operacji

❏ Time travel nie kosztuje dodatkowego storage, korzysta z tych samych plików Parquet co bieżąca wersja

Time travel w praktyce

Time travel w Delta Lake jest używany na co dzień, nie tylko do audytu, ale też do szybkiej reakcji na błędy i analizy trendów.

Zastosowania:

Audyt zmian w danych

Przywracanie stanu po błędnej operacji

Porównywanie stanu danych między wersjami

Debugowanie pipeline'ów

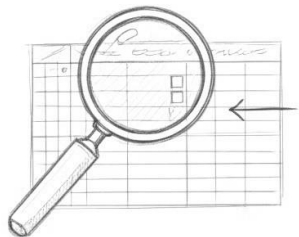
Ograniczenia:

- Spadek wydajności przy bardzo starych wersjach
- Koszt przechowywania wersji historycznych
- Historia to domyślnie 30 dni, ale tylko dopóki VACUUM nie usunie plików (domyślnie 7 dni)

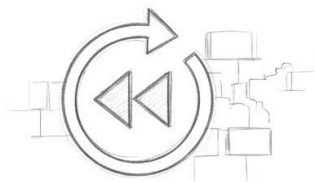
Przykłady:

```
-- Porównanie danych między wersjami
SELECT COUNT(*) as current_count FROM sales_table;
SELECT COUNT(*) as yesterday_count
FROM sales_table TIMESTAMP AS OF '2024-01-14';

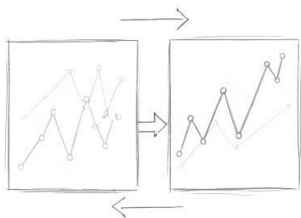
-- Szukanie różnic
SELECT * FROM sales_table
EXCEPT
SELECT * FROM sales_table VERSION AS OF 1;
```



DATA AUDITING



ROLLBACKS



VERSION COMPARISONS



DATA PIPELINE
DEBUGGING

RESTORE, cofanie zmian

-- Przywrócenie konkretnej wersji

```
RESTORE TABLE orders TO VERSION AS OF 2
```

-- Przywrócenie stanu z danej chwili

```
RESTORE TABLE orders TO TIMESTAMP AS OF '2024-01-10 08:00:00'
```

RESTORE tworzy nową wersję

Nie kasuje historii. Sama operacja RESTORE zapisuje się jako nowa wersja, w pełni audytowalna. Historia tabeli zostaje.

Szybszy niż backup

Po przypadkowym DELETE albo OVERWRITE polecenie RESTORE działa rzędy wielkości szybciej niż klasyczny backup.

Ograniczenie retencji

Przywrócisz tylko wersje, których pliki nadal istnieją. Jeśli VACUUM usunął stare pliki, tych wersji już nie odzyskasz.

Jak utrzymać szybkie tabele Delta

Wiele małych plików spowalnia każde zapytanie. Delta ma na to trzy odpowiedzi, a na tabelach managed Databricks uruchamia je za Ciebie:

OPTIMIZE

Scala małe pliki w większe. Uruchom po intensywnych zapisach albo zostaw decyzję predictive optimization.

VACUUM

Usuwa pliki, do których nie odwołuje się żadna żywa wersja. Domyślna retencja to 7 dni i tyle samo wynosi zasięg time travel.

Liquid clustering

CLUSTER BY zastępuje partycjonowanie i ZORDER. CLUSTER BY AUTO pozwala Databricks dobrać klucze na podstawie historii zapytań.

Predictive optimization

Na tabelach managed w Unity Catalog Databricks sam uruchamia OPTIMIZE, VACUUM i ANALYZE na compute serverless, a pomija je, gdy się nie opłacają.

Co nadal zależy od Ciebie

Klucze klastrowania dla bardzo dużych tabel, retencja inna niż domyślna i to, czy tabela ma być managed. Tabele external utrzymujesz sam.

SEKCJA 6

Orkiestracja w Lakeflow

Od ręcznego uruchamiania notebooków do pewnych, monitorowanych pipeline'ów produkcyjnych.

Po co nam orkiestracja?

Problem: ręczne uruchamianie

Ręczne odpalanie notebooków nie daje powtarzalności, śladu audytowego, alertów ani zależności. Jeden pominięty bieg to nieaktualne dane na produkcji.

Rozwiązanie: Lakeflow Jobs

Lakeflow Job to zadeklarowany DAG tasków z harmonogramem i monitoringiem. Sam obsługuje ponowienia, zależności, repair run i śledzenie kosztów.

W porównaniu z cronem

Lakeflow Jobs dokłada zależności między taskami, równoległość, ponowienia per task i repair run. Tego cron po prostu nie ma.

Ekonomia job clustera

Serverless albo job cluster startuje na czas biegu i gaśnie zaraz po nim. Zero kosztu bezczynności, dlatego produkcja nie chodzi na all-purpose.

Pełna audytowalność

Każdy bieg jest zapisany: czas startu i końca, koszt, status i wynik. Odtworzysz dokładnie, co i kiedy się wykonało.

Budowa Lakeflow Joba



Task

Najmniejsza jednostka pracy: notebook, skrypt Python, zapytanie SQL, projekt dbt albo pipeline Lakeflow. Każdy task ma własną konfigurację i status.



Depends on

Krawędź grafu. Task Silver rusza dopiero po udanym Bronze. Awaria zatrzymuje kaskadę, kolejne taski są pomijane, a nie psute.



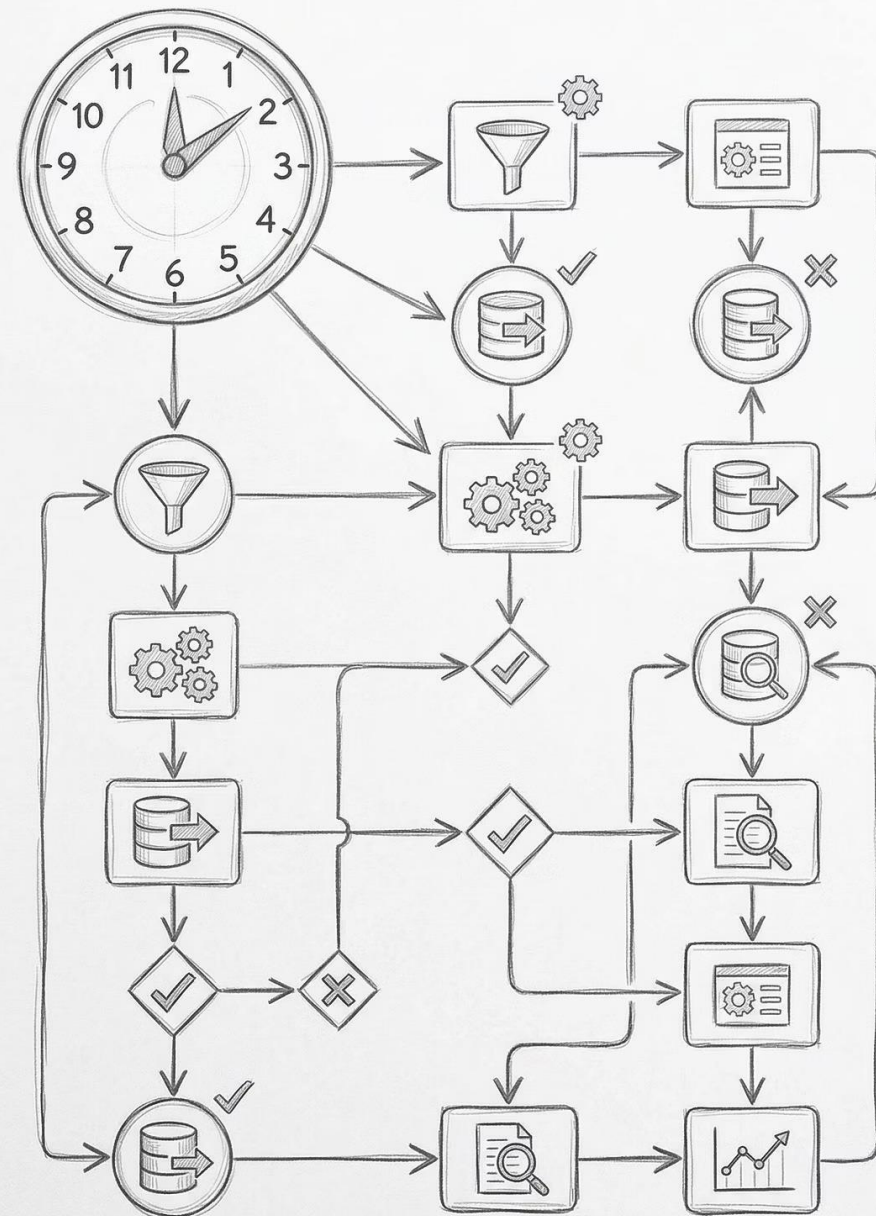
Parametry

Przekazujesz je do notebooków przez `dbutils.widgets` lub `sys.argv`. Dzięki temu pipeline działa dla różnych dat, środowisk i źródeł.



Ponowienia

Automatyczne ponowienie po błędzie, konfigurowane per task. Krytyczny task może próbować 3 razy, idempotentny raz.



Harmonogram i monitoring

Składnia CRON Quartz

```
0 0 6 * * ?
```

= codziennie o 06:00

Rodzaje wyzwalaczy

- **Manual**, uruchomienie na żądanie
- **CRON**, harmonogram czasowy
- **File arrival**, start po nowym pliku
- **Continuous**, zadania strumieniowe

Repair run

Powtarza **tylko taski, które padły**, a nie cały pipeline. Gdy Bronze się udał, a Silver nie, repair run rusza od Silver. Oszczędza czas i koszt compute.

Alerty

Powiadomienia mailem lub webhookiem przy starcie, sukcesie, błędzie i pominięciu. Ustawisz je per job albo per task.

Panel metryk

Czas wykonania, koszt i historia ostatnich biegów, wszystko w interfejsie Jobs.

Alternatywa deklaratywna: Lakeflow pipelines

Job orkiestruje notebooki, które napisałeś. Pipeline odwraca podejście: deklarujesz tabele, a Databricks ustala kolejność, logikę przyrostową i utrzymanie.

```
from pyspark import pipelines as dp

@dp.table(name="silver_orders")
@dp.expect_or_drop("valid_id", "order_id IS NOT NULL")
def silver_orders():
    return spark.readStream.table("bronze_orders")
```

Co dostajesz

Zależności wyprowadzone z kodu, odświeżanie przyrostowe, expectations dla jakości danych, automatyczne ponowienia i graf lineage całego przepływu.

Co kiedy wybrać

Pipeline pasuje do ciągłego ETL do warstw medalionu. Job pasuje do reszty: notebooki ad hoc, dbt, SQL, machine learning albo wywołanie pipeline jako jednego tasku.

 Delta Live Tables to stara nazwa tej funkcji, a `APPLY CHANGES INTO` to dziś `AUTO CDC INTO`. W starszych materiałach spotkasz obie formy.

SEKCJA 7

Unity Catalog i AI/BI

Governance dla firmy, lineage i wbudowana analityka, wszystko w Databricks.

Unity Catalog, governance dla całej organizacji

Jeden katalog dla wszystkich workspace

Przed Unity Catalog każdy workspace miał własny metastore Hive, co tworzyło silosy. UC daje jeden wspólny katalog dla całego konta Databricks.

Lineage danych

Automatyczne śledzenie przepływu danych między tabelami, także na poziomie kolumn. Wiesz, skąd pole pochodzi i dokąd trafia.

Log audytowy

Każdy SELECT, INSERT i GRANT jest zapisany: kto, kiedy i co. Niezbędne przy zgodności, incydentach i audycie RODO.

Tagi i komentarze

Dokumentujesz tabele i kolumny wprost w katalogu, bez osobnej wiki. Tagi pomagają wyszukiwać i klasyfikować dane na skalę.

Model uprawnień

```
GRANT SELECT ON TABLE catalog.schema.orders TO `analyst@company.com`;  
GRANT USE SCHEMA ON SCHEMA catalog.bronze TO `data_engineers`;  
GRANT ALL PRIVILEGES ON CATALOG dev TO `admin_group`;
```

Hierarchia uprawnień

Uprawnienia kaskadują: `USE CATALOG` → `USE SCHEMA` → `SELECT` → `MODIFY` → `CREATE TABLE` → `ALL PRIVILEGES`.
Nadawaj je na właściwym poziomie.

Ochrona wierszy i kolumn

Filtrujesz dane w czasie zapytania, bez osobnych widoków. Użytkownik widzi tylko dozwolone wiersze i kolumny, pilnuje tego katalog.

Service principals

Zadania automatyczne nigdy nie powinny chodzić na koncie osobowym. Używaj service principals, czyli tożsamości dla pipeline'ów z minimalnymi uprawnieniami.

Dashboardy AI/BI

Natywne dashboardy w Databricks

Budujesz i udostępniasz dashboardy wprost w platformie, bez eksportu do Tableau czy Power BI. Źródłem jest dowolna tabela Delta lub wynik SQL.

Odświeżanie

- Ręcznie, na żądanie
- Z harmonogramu CRON
- Na żywo przy otwarciu dashboardu

Typy wykresów

Słupki, linie, punkty, kołowe, liczniki, heatmapy i mapy. Metric views w Unity Catalog pozwalają zdefiniować KPI raz i używać go wszędzie.

Genie, asystent AI

Użytkownik pyta w języku naturalnym. Genie generuje SQL, wykonuje go i pokazuje wynik na wykresie. Znajomość SQL nie jest potrzebna.

Udostępnianie dashboardów

Udostępniasz linkiem. Przy odpowiedniej konfiguracji odbiorca nie potrzebuje konta Databricks, co jest wygodne dla biznesu spoza platformy.

SEKCJA 8

Projekt końcowy: pipeline RetailHub

Składamy wszystko w całość i budujemy kompletny pipeline produkcyjny, od surowych zamówień po dzienny raport biznesowy.

Scenariusz biznesowy

Firma

RetailHub obsługuje zamówienia z wielu kanałów sprzedaży. Zamówienia przychodzą w plikach JSON, jeden plik na dzień.

Cel

Codziennie o **06:00** automatycznie odświeżamy dzienny raport przychodów w tabeli Gold, tak aby czekał na analityków od rana.

Dane wejściowe

`orders_batch.json` w volume Unity Catalog, surowe zamówienia z jednego dnia. Zawiera identyfikatory, kwoty, rabaty, znaczniki czasu i kanał sprzedaży.

Oczekiwany wynik

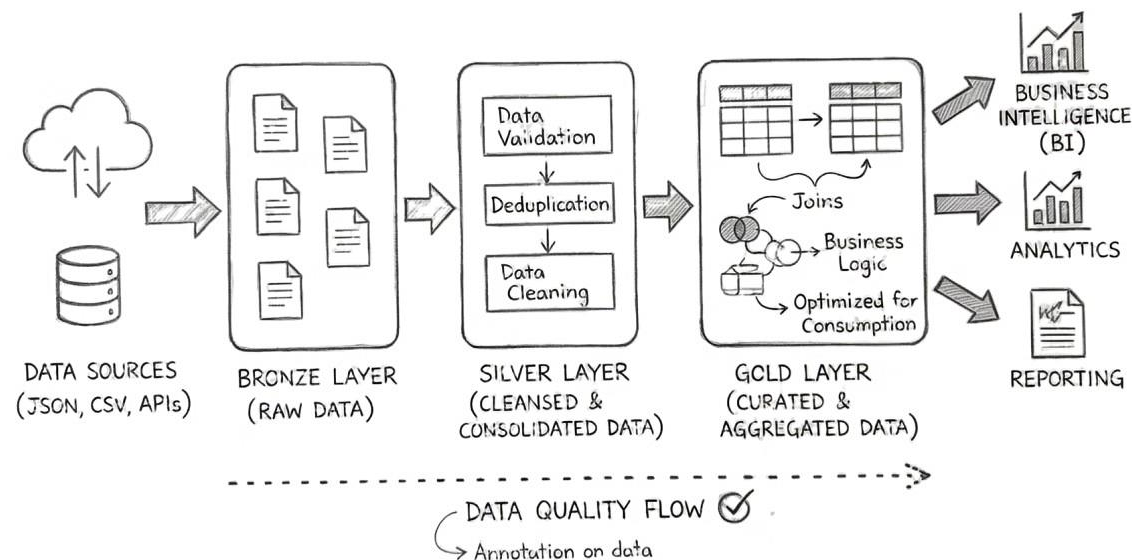
Tabela `gold.lab_daily_orders`, jeden wiersz na dzień i cztery kolumny: data, liczba zamówień, przychód i średnia wartość zamówienia.

Ograniczenie

Pipeline musi być idempotentny, dwa biegi dla tego samego dnia dają ten sam wynik. Żadnych duplikatów w Gold.

Architektura pipeline

Każdy etap to osobny notebook i osobna tabela Delta. Całość orkiestruje Lakeflow Job z jawnymi zależnościami: Bronze musi skończyć przed Silver, a Silver przed Gold.



Schemat tabeli Gold

Kolumna	Typ	Opis
order_date	DateType	Dzień agregacji, jeden wiersz na datę
order_count	LongType	Liczba zamówień danego dnia
total_revenue	DoubleType	Suma net_amount, czyli po rabacie
avg_order_value	DoubleType	Średnia wartość zamówienia, zaokrąglona do 2 miejsc

-  Schemat Gold jest celowo minimalny i stabilny. Raporty i dashboardy zakładają, że się nie zmienia. Każda zmiana łamie zgodność, więc uzgodnij ją z odbiorcami.

Weryfikacja po uruchomieniu joba

-- Czy dane trafiły do tabeli?

```
SELECT * FROM gold.lab_daily_orders  
ORDER BY order_date DESC  
LIMIT 10;
```

-- Ile dni jest pokrytych?

```
SELECT COUNT(*) AS total_days  
FROM gold.lab_daily_orders;
```

-- Kontrola przychodu

```
SELECT  
  SUM(total_revenue),  
  MIN(order_date),  
  MAX(order_date)  
FROM gold.lab_daily_orders;
```

-- Audyt: jakie operacje wykonano?

```
DESCRIBE HISTORY gold.lab_daily_orders;
```

Te cztery zapytania to standardowa lista kontrolna po biegu. Na dev uruchamiaj je po każdym wykonaniu, a na produkcji włącz do automatycznych kontroli jakości danych.

SEKCJA 9

Podsumowanie

Przypominamy, co dziś zbudowaliśmy, i planujemy dalsze kroki.

Co dziś zbudowaliśmy

Warstwa	Notebook	Kluczowa technika
Bronze	lab_task_01_bronze	StructType, StringType, kolumny metadanych
Silver	lab_task_02_silver	cast, to_timestamp, when, filtrowanie nulli
Gold	lab_task_03_gold	groupBy, agg, to_date, round
Orkiestracja	WORKSHOP_final_project	Lakeflow Job, CRON, notebook.exit

W jeden dzień przeszliśmy od surowych plików JSON do w pełni zorkiestrowanego pipeline'u z harmonogramem, gwarancjami ACID, time travel i codziennym odświeżaniem. Dokładnie tak wygląda to we wdrożeniach produkcyjnych.

Co warto zapamiętać

Lakehouse

Nie wybieraj między hurtownią a data lake, Databricks daje jedno i drugie. Elastyczny storage z transakcjami i governance klasy enterprise.

Delta

ACID, MERGE i time travel to standard, a tabele managed same się optymalizują. Nie używaj gołego Parquet tam, gdzie będą zmiany lub audyt.

Job cluster

Produkcja działa na serverless albo job clusterze. All-purpose jest do pracy interaktywnej, nie zostawiaj go włączonego bez potrzeby.

Medalion

Trzy warstwy to trzy kontrakty jakości. Bronze obiecuje komplet danych surowych, Silver czystość, a Gold poprawność biznesową.

Jawny schemat

Zgadywany schemat to dług techniczny. Zdefiniuj StructType z góry, ochroni pipeline przed zmianami w źródle i pokaże błędy od razu.

Unity Catalog

Governance na poziomie kolumn, nie tylko tabel. Jeden katalog, jeden model uprawnień i pełny lineage dla całej organizacji.

Co dalej

Ćwicz za darmo

Databricks Free Edition daje darmowy workspace serverless z notebookami, SQL, pipeline'ami i dashboardami. Zastąpił wycofane Community Edition.

Zbuduj projekt jeszcze raz

Uruchom pipeline RetailHub na własnych danych. Zmień plik źródłowy, dodaj kolumnę, zepsuj coś celowo i przywróć.

Ścieżka certyfikacji

Databricks Certified Data Engineer Associate to naturalny kolejny krok. Przewodnik po egzaminie mówi dokładnie, co obejmuje.

Pogłębiaj platformę

Kolejne tematy to governance w Unity Catalog, Lakeflow pipelines, streaming i strojenie wydajności. Każdy to osobne szkolenie.

Czytaj źródło

docs.databricks.com jest aktualna i dobrze się przeszukuje. Release notes pokazują, co zmienia się w platformie każdego miesiąca.

Pytaj asystenta

Wbudowany asystent wyjaśnia błędy i pisze szkielet kodu. Traktuj go jak parę do kodowania i sprawdzaj wynik w dokumentacji.

Pytania i feedback

Otwarte pytania

To dobry moment na pytania, które pojawiły się w ciągu dnia, o platformę, pipeline, architekturę albo cokolwiek ze szkolenia.

Żadne pytanie nie jest zbyt podstawowe. Jeśli o czymś myślisz, ktoś na sali pewnie też.

Ankieta

Poświęć 2 minuty na ankietę, Twoje uwagi wprost kształtują kolejne edycje tego szkolenia.

Link albo kod QR pokażę teraz na ekranie.

Kontakt

Po szkoleniu możesz pisać do prowadzącego z pytaniami. Dane kontaktowe są na końcu ankiety.



Dziękuję za uwagę, pytania i energię przez cały dzień. Powodzenia w projektach! 🚀